CS1010 Programming Methodology

Lecture Notes

Seojin

Contents

L0: Introduction **2**
0.0 Programming . . . 2
0.1 Basic Program . . . 3
0.2 Variables and data storage . . . 3
0.3 Arithmetic and Assignment Operators . . . 6
0.4 Ternary Operators . . . 8
0.5 Undefined Behaviour . . . 9
0.X Lab . . . 11

L1: Conditionals and Functions **11**
1.1 Abstraction I: Functions . . . 11
1.2 Conditional Statements . . . 12
1.X Lab . . . 14

L2: Abstractions, Structs, and the Stack **14**
2.1 Abstraction II: Functions . . . 14
2.2 The Stack and Block Scoping . . . 15
2.X Lab . . . 16

L3: Fixed Size arrays and Recursion **18**
3.1 Arrays . . . 18
3.2 Arrays and Structs . . . 21
3.3 Multidimensional Arrays . . . 22
3.4 Recursion . . . 22
3.L Lab . . . 24

L4: Loops **24**
4.1 `while` Loops and `for` Loops . . . 24
4.2 Loop control with `break`, `continue`, `return` . . . 26
4.3 Array recursion . . . 27
4.4 Examples . . . 27

L5: Pointers and Strings **27**
5.1 Pointers . . . 27
5.2 Address-of (&) and Dereferencing (*), nullptr, pointers to pointers . . . 29
5.3 Arrays, Pointer Arithmetic, `sizeof`, Structs, Functions . . . 31
5.4 Strings . . . 35
5.4 Strings . . . 36
5.5 Pointer Provenance . . . 38

L6: Dynamic memory allocation **41**

Admin matters:

Instructors:

- Dr. Eldon Chung
- Dr. Sriram Sami

Exam Schedule:

Activity	Date	Time
Practical Exam	Tue, 15 Sep	6pm-9pm
Midterm	Mon, 28 Sep	4pm-6pm
Practical Exam	Tue, 13 Oct	6pm-9pm
Practical Exam	Tue, 10 Nov	6pm-9pm
Final	Wed, 25 Nov	9am-11am

More info: CS1010 website

L0: Introduction

0.0 Programming

Program
: is a set of instructions that a computer runs.

Programming
: is writing code that is used to create a program.

More specifically,

1. The **CPU** (Central Processing Unit) reads information from the computer memory.
2. The CPU then writes information to the *Program Code*. The program code that we make is stored in the memory.
3. The Program stores data (Program data) for intermediate steps. These are also stored in memory.

- We write in a higher-level programming language like C to create instructions that the CPU has to execute. A **compiler** (e.g. `clang` or `gcc`) takes the C code and produces assembly code.
- We use high-level language and compilers so that our code is *portable*; we leave lower level details to be handled by the compiler.
- We also leave hardware level optimisations to be handled by compilers.

C
: is a *compiled* programming language.

Compiler
: has to take C code written to produce assembly instructions that the CPU will execute.

Pros	Cons
lots of control: imperative, low level	More work required by programmer
Simple language	Many footguns
little to no overhead	

Nevertheless, C is:

- Low-level enough for hardware (firmware)
- Widely supported across embedded systems

- Pervasive in system code
- Good starter language.

0.1 Basic Program

Anatomy:

```
#include <stdio.h>
int main(void) {
    printf("Hello world!\n");
    return 0;
}
```

1. `#include <stdio.h>` makes the C "standard input/output" header file available to this program.
2. `int main(void) {` declares the main function, and code **starts** executing from within `main`. The `void` means that the function does not take any input arguments. The opening brace `{` means the start of the code within `main`.
3. `printf("Hello world!\n");` Creates a string `"Hello world!"` and a new line character `\n` and passes the argument to `printf`, which is sent to the standard output of the program
4. Semicolon `;` indicates the end of the statement.
5. Closing brace `}` ends the code within `main`.

Procedure:

1. Create file with extension .c, (*e.g.* `hello.c`)
2. Write C code inside.
3. Compile C code to an executable program\

```
clang -o hello hello.c
```

4. Run if no errors.

```
./hello
```

0.2 Variables and data storage

0.2.1 Variables

Variables:

Type name(s)	Min value	Max value	Size (bytes)
`char`	-128	+127	1 (8 bits)
`unsigned char`	0	+255	1 (8 bits)
`int`	-32768 (-2^{15})	+32767 ($2^{15}-1$)	2
`unsigned int`	0	+65535 ($2^{16}-1$)	2
`long`	-2147483648 (2^{31})	+2147483647 ($2^{31}-1$)	4
`unsigned long`	0	+4294967295 ($2^{32}-1$)	4
`unsigned long long`	0	$2^{64}-1$	8
`float`	6-7 d.p.	6-7 d.p.	4
`double`	15-17 d.p.	15-17 d.p.	8

When to use which type

- `char` for ASCII characters
- `int` for default integral types
- `unsigned int` if integer > 0
- `float` if you want to save memory and use decimals.
- `double` if your values might be more precise decimal. Use double and instead of float, unless with good reason.

Initialization You can also initialize a variable without declaring its value.

```
int a;
int b;
a = 372;
b = a - 200;
```

Overflow If you have an `unsigned int` number at 255, and you increment it, you'll get 256 in return. As expected. If you have an `unsigned char` number at 255, and you increment it, you'll get 0 in return. It resets starting from the initial possible value.

If you have a `unsigned char` number at 255 and you add 10 to it, you'll get the number 9:

```
#include <stdio.h>

int main(void) {
    unsigned char j = 255;
    j = j + 10;
    printf("%u", j); /* 9 */
}
```

If you don't have a signed value, the behavior is undefined. It will basically give you a huge number which can vary, like in this case:

```
#include <stdio.h>

int main(void) {
    char j = 127;
    j = j + 10;
    printf("%u", j); /* 4294967177 */
}
```

In other words, C does not protect you from going over the limits of a type. You need to take care of this yourself. However, if you initialize a variable with a wrong value, you will likely receive a warning:

```
#include <stdio.h>

int main(void) {
    char j = 1000;
    char i;
    i = 9999;
    // This will give warning.
}
```

However, incrementing the number will not give a warning and overflow.

```
#include <stdio.h>

int main(void) {
    char j = 0;
    j += 1000;
}
```

ASCII table:

Dec	Symbol	Dec	Symbol	Dec	Symbol	Dec	Symbol	Dec	Symbol
48	0	63	?	78	N	93	]	108	l
49	1	64	@	79	O	94	^	109	m
50	2	65	A	80	P	95	_	110	n
51	3	66	B	81	Q	96	`	111	o
52	4	67	C	82	R	97	a	112	p
53	5	68	D	83	S	98	b	113	q
54	6	69	E	84	T	99	c	114	r
55	7	70	F	85	U	100	d	115	s
56	8	71	G	86	V	101	e	116	t
57	9	72	H	87	W	102	f	117	u
58	:	73	I	88	X	103	g	118	v
59	;	74	J	89	Y	104	h	119	w
60	<	75	K	90	Z	105	i	120	x
61	=	76	L	91	[	106	j	121	y
62	>	77	M	92	\	107	k	122	z

It is possible to find the size of a variable.

sizeof()

```c
#include <stdio.h>
int main(void) {
    int myInt;
    float myFloat;
    double myDouble;
    char myChar;
    printf("%d\n", sizeof(myInt));
    printf("%d\n", sizeof(myFloat));
    printf("%d\n", sizeof(myDouble));
    printf("%d\n", sizeof(myChar));
    return 0;
}
```

output:

```
4
4
8
1
```

0.2.2 Format Specifiers

Format specifiers

Specifier	Output	Example
d or i	Signed decimal integer	392
u	Unsigned decimal integer	7235
o	Unsigned octal	610
x	Unsigned hexadecimal integer	7fa
X	Unsigned hexadecimal integer (uppercase)	7FA
f	Decimal floating point, lowercase	392.65
F	Decimal floating point, uppercase	392.65
e	Scientific notation (mantissa/exponent), lowercase	3.9265e+2
E	Scientific notation (mantissa/exponent), uppercase	3.9265E+2

Specifier	Output	Example
g	Shortest representation: %e or %f	392.65
G	Shortest representation: %E or %F	392.65
a	Hexadecimal floating point, lowercase	-0xc.90fep-2
A	Hexadecimal floating point, uppercase	-0XC.90FEP-2
c	Character	a
s	String of characters	sample
p	Pointer address	b8000000
n	Nothing printed. Argument must be a pointer to a signed int; the number of characters written so far is stored there.	—
%	A literal % character	%

Below is an example:

```c
#include <stdio.h>

int main(void) {
    int sixtyseven = 67;
    double pi = 3.1415926535897;
    char ahh = 'a';
    char pain[] = "what?"; //adding [] to the end of the variable will make it a string instead of
   a char
    printf("Integer: %d\n", sixtyseven);
    printf("2dp Float: %.2f\n", pi);
    printf("6dp Float: %.6f\n", pi);
    printf("Char: %c\n", ahh);
    printf("String: %s\n", pain);

    return 0;
}
```

Integer Truncation When you output the value of `double` with `%d`, you will get the *floor* of the value, truncating the decimal point and leaving the integer part.

```c
int x = 5;
x = x + 0.1; // Behind the scenes: (int)((double)5.0 + 0.1)
printf("%d", x);
```

0.3 Arithmetic and Assignment Operators

0.3.1 Arithmetic Operators

Arithmetic operators let us do math:

Operator	Meaning	Example
=	Assignment	x=1
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulo (remainder)	x % y

0.3.2 Unary operators

Unary operators only take one operand:

Operator	Name	Example
`+`	Unary plus	`+a`
`-`	Unary minus	`-a`
`++`	Increment	`a++` or `++a`
`--`	Decrement	`a--` or `--a`

Example:

```
int a = 2;
int b;
b = a++ /* b is 2, a is 3 */
b = ++a /* b is 4, a is 4 */
b = a-- /* b is 4, a is 3 */
b = --a /* b is 2, a is 2 */
```

0.3.3 Comparison Operators

Comparison Operators

Operator	Name	Example
`==`	Equal operator	`a == b`
`!=`	Not equal operator	`a != b`
`>`	Bigger than	`a > b`
`<`	Less than	`a < b`
`>=`	Bigger than or equal to	`a >= b`
`<=`	Less than or equal to	`a <= b`

0.3.4 Logic Operators

Logical operators: work with boolean values.

Operator	Name	example
`!`	NOT	`!a`
`&`	AND	`a && b`
`|`	OR	`a || b`

0.3.5 Assignment Operators

Assignment Operators perform arithmetic transformation and and assignment

Operator	Name	Example
`+=`	Addition assignment	`a += b`
`-=`	Subtraction assignment	`a -= b`
`*=`	Multiplication assignment	`a *= b`
`/=`	Division assignment	`a /= b`
`%=`	Modulo assignment	`a %= b`

0.3.6 Integer Division

If `x` and `y` are **integer-like types**, **`/` truncates the fractional part**.

```
printf("%d\n", 50 / 20);  // 2
printf("%d\n", 50 % 20);  // 10
```

- `50 / 20` → 2, not 2.5
- `50 % 20` → 10 (remainder)
- `%` only works on **integral types**; not `float`/`double`.

0.3.7 Division with `double`

If you want to preserve decimal values, make sure **at least one operand is `double`/`float`**:

```
int x = 50;

x / 20       // 2
x / 20.0     // 2.5
50.0 / 20.0  // 2.5
(double)x / 20  // 2.5
```

You can explicitly convert a value using a **cast**:

```
(double)x
```

0.3.8 Operator Precedence

- `*`, `/`, `%` have **higher priority** than `+` and `-`
- `()` has the **highest priority**
- Similar to the normal order of operations in mathematics.

```
x + 3 / x * (3 + x)
```

is evaluated according to these precedence rules.

> **Important:** `=` means **assignment**, not mathematical equality.

```
int x = 5;
x = x + 2;  // x is now 7
x += 2;     // x is now 9
```

Assignment **changes the state of the program** by changing the value stored in a variable.

0.4 Ternary Operators

The **ternary operator** provides a compact way to choose between two expressions based on a condition.

```
<expression1> ? <expression2> : <expression3>;
```

In English:

> If `<expression1>` is true, evaluate `<expression2>`; otherwise, evaluate `<expression3>`.

Example:

```
double product_cost = 888;

double discount_amt = (product_cost >= 670) ? 2 : 0;
```

Since `product_cost` is 888:

```
888 >= 670 → true
```

Therefore:

```
discount_amt = 2
```

If `product_cost` were less than 670:

```
discount_amt = 0
```

Common condition tests

Operator	Meaning
`>`	More than
`>=`	More than or equal to
`<`	Less than
`<=`	Less than or equal to
`==`	Equal to
`!=`	Not equal to

Another example:

```
(product_cost >= 670)
    ? printf("You've reached min-spend")
    : printf("Spend another %f cents\n", 670 - product_cost);
```

The ternary operator is useful for **simple conditions** where there are two possible outcomes.

0.5 Undefined Behaviour

Undefined Behaviour (UB) is a major recurring aspect of C.

C is defined by a document called the **C Standard**. CS1010 uses the **C23 standard**.

The standard specifies what should happen for valid C code. However, for some erroneous code, the standard **imposes no requirements**.

If a program contains **undefined behaviour**:

> The program is allowed to do anything.

It might:

- produce an unexpected result
- crash
- appear to work
- behave differently on another compiler/system
- do something else entirely

There are no guarantees.

Example: Division by zero

```c
#include <stdio.h>

int main() {
    printf("%d\n", 5);
    printf("%d\n", 1 / 0);
    return 0;
}
```

The expression:

```
1 / 0
```

has **undefined behaviour**.

Do **not** assume that it will always produce a particular result.

Integer Overflow

Overflow occurs when a value exceeds the range that its type can represent.

Assuming `int` is 4 bytes (32 bits):

Unsigned integer overflow is well-defined:

```c
unsigned int c = 4294967295;
c = c + 1;
```

The value **wraps around**:

```
c = 0
```

Signed integer overflow is undefined behaviour:

```c
int c = 2147483647;
c = c + 1;  // Undefined Behaviour!
```

So:

Situation	Behaviour
Unsigned integer overflow	Well-defined; wraps around
Signed integer overflow	**Undefined Behaviour**

Key takeaway

UB = the C Standard gives no requirements for what happens.

Therefore, **never rely on what seems to happen when testing UB**. A program may behave differently depending on the compiler, optimisation, machine, or other circumstances.

UB is an unavoidable part of C, so learning to recognise and avoid it is important.

Compiler Warnings

The compiler can warn you about **some** problematic cases.

```
Listen to what your compiler says!
```

Compiler warnings are useful for catching potential problems before they become bugs.

0.X Lab

Goal: Practice working with variables, format specifiers, and the ternary operator.

Exercise 1 — Temperature conversion

Write a program that stores a temperature in Celsius as a `double`, converts it to Fahrenheit, and prints both values to 2 decimal places.

```c
#include <stdio.h>

int main(void) {
    double celsius = 30.0;
    double fahrenheit = celsius * 9.0 / 5.0 + 32;
    printf("%.2f C = %.2f F\n", celsius, fahrenheit);
    return 0;
}
```

Exercise 2 — Even or odd, with a ternary

Read an `int` and use the ternary operator (not `if`) to print whether it is even or odd.

```c
#include <stdio.h>

int main(void) {
    int n = 7;
    printf("%s\n", (n % 2 == 0) ? "even" : "odd");
    return 0;
}
```

Things to watch out for

- Integer division silently truncates — cast to `double` if you need a fractional result (see 0.3.7).
- `%` only works on integer types.
- Always double check your format specifier matches the variable's type (e.g. `%f` for `double`/`float`, `%d` for `int`) — mismatching them is undefined behaviour (see 0.5).

L1: Conditionals and Functions

1.1 Abstraction I: Functions

Function
: is a named, reusable block of code that performs a specific task. It takes zero or more **arguments** as input, and can **return** a single value as output.

Why functions?

- **Abstraction** — the caller only needs to know *what* a function does (its interface), not *how* it does it (its implementation).
- **Reuse** — write the logic once, call it many times.
- **Readability** — breaking a program into small, well-named functions makes it easier to follow.

Anatomy of a function

```c
int add(int a, int b) {
    int sum = a + b;
    return sum;
}
```

- `int` (leftmost) is the **return type** — the type of value the function sends back.
- `add` is the function's **name**.
- `(int a, int b)` are the **parameters** — the inputs the function expects, along with their types.
- `return sum;` sends the value of `sum` back to whoever called the function. A function with return type `void` does not return a value, and can use a bare `return;` (or nothing) to end early.

Calling a function

```
int result = add(3, 4); // result is 7
```

Function prototypes

If a function is defined *after* `main`, or in another file, the compiler needs to know its signature before it's used. A **prototype** declares the signature without the body:

```
int add(int a, int b); // prototype

int main(void) {
    int result = add(3, 4);
    return 0;
}

int add(int a, int b) { // definition
    return a + b;
}
```

1.2 Conditional Statements

1.2.1 If/else constructs

The `if` statement runs a block of code only when a condition is true.

```
if (condition) {
    // runs when condition is true
}
```

Add an `else` to handle the opposite case:

```
if (condition) {
    // condition is true
} else {
    // condition is false
}
```

Chain multiple conditions with `else if`:

```
if (score >= 90) {
    printf("A\n");
} else if (score >= 75) {
    printf("B\n");
} else if (score >= 50) {
    printf("C\n");
} else {
    printf("F\n");
}
```

Only the **first** branch whose condition is true runs; the rest are skipped.

Always use braces

As seen in the L2 lab below, omitting `{ }` means only the *single statement* immediately after the

`if/else` belongs to it — everything after that runs unconditionally regardless of the condition. Always wrap the body in `{ }`, even for one-line bodies, to avoid this class of bug.

1.2.2 Comparison and logical operators

Recall the comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) and logical operators (`&&`, `||`, `!`) from 0.3. Inside an `if`, these are combined to build more complex conditions.

Truth tables

a	b	a && b	a || b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

a	!a
0	1
1	0

Short-circuit evaluation

`&&` and `||` evaluate left to right and stop early once the result is already decided:

- `a && b` — if `a` is false, `b` is **never evaluated** (the whole expression is already false).
- `a || b` — if `a` is true, `b` is **never evaluated** (the whole expression is already true).

This matters when the right-hand side has side effects (e.g. a function call that prints something), as seen in the `foo() && bar()` example in the L2 lab below.

1.2.3 Boolean operators

C did not have a built-in boolean type until relatively recently. `<stdbool.h>` provides one:

```c
#include <stdbool.h>

bool is_valid = true;
```

Under the hood:

- `bool` is really a small integer type; `true` is `1` and `false` is `0`.
- Any non-zero value is treated as "true" in a condition; only `0` is "false".

```c
int x = 5;
if (x) {        // true, since x != 0
    printf("x is truthy\n");
}
```

Returning a `bool` from a function

```c
#include <stdbool.h>

bool is_even(int n) {
    return n % 2 == 0;
}
```

This is the pattern used by `foo()` and `bar()` in the L2 lab below — functions that return `bool` and are combined with `&&/||` inside an `if`.

1.X Lab

Goal: Practice writing your own functions that use conditionals.

Exercise — Grade classifier

Write a function `char grade(int score)` that returns `'A'`, `'B'`, `'C'`, or `'F'` based on the boundaries used in 1.2.1, then call it from `main` and print the result.

```c
#include <stdio.h>

char grade(int score) {
    if (score >= 90) {
        return 'A';
    } else if (score >= 75) {
        return 'B';
    } else if (score >= 50) {
        return 'C';
    } else {
        return 'F';
    }
}

int main(void) {
    int score = 82;
    printf("Grade: %c\n", grade(score));
    return 0;
}
```

Stretch goal

Write `bool is_leap_year(int year)` using the rule: divisible by 4, except centuries, unless also divisible by 400. Combine `&&`, `||`, and `!` from 1.2.2.

L2: Abstractions, Structs, and the Stack

2.1 Abstraction II: Functions

2.1.1 Abstraction of Behavior: Functions

Section 1.1 introduced functions as reusable blocks of code. Here we look at *why* wrapping behaviour in a function is a form of **abstraction**.

When we call a function, we only need to know:

1. Its **name**
2. Its **parameters** (types and meaning)
3. Its **return type**

We don't need to know *how* it's implemented internally — that's hidden inside the function body. This is exactly what happens with `read_point()` and `print_movement()` in the lab below: `main` doesn't need to know how a point is read or how movement is computed, only that these functions exist and what they do.

Pass-by-value

In C, arguments are passed **by value** — the function receives a *copy* of the argument. Modifying a parameter inside a function does not affect the caller's variable.

```c
void increment(int x) {
    x = x + 1; // only changes the local copy
}
```

```c
int main(void) {
    int n = 5;
    increment(n);
    printf("%d\n", n); // still 5
}
```

2.1.2 Abstraction of Data: Structs

Just as functions abstract *behaviour*, **structs** abstract *data* — they let us group related variables into a single named type.

```c
typedef struct {
    int x;
    int y;
} Point;
```

- `struct { ... }` defines a new structure with the listed fields (**members**).
- `typedef ... Point;` gives the struct the alias `Point`, so we can write `Point p;` instead of `struct { ... } p;` every time.

Creating and accessing a struct

```c
Point p;
p.x = 3;
p.y = 4;
printf("(%d, %d)\n", p.x, p.y); // (3, 4)
```

The `.` (dot) operator accesses a member of a struct.

Why this is abstraction

Without a struct, a point would be two separate, unrelated `int` variables (`px`, `py`), and a function relating two points would need four separate parameters. With `Point`, related data travels together as one value — as seen in the lab below, where `print_movement(Point start, Point end)` takes just two arguments instead of four.

2.2 The Stack and Block Scoping

2.2.1 Stack Frames

Every time a function is called, the program sets aside a small block of memory called a **stack frame** to hold that call's local variables, parameters, and return address.

- When a function is **called**, a new frame is **pushed** onto the **call stack**.
- When the function **returns**, its frame is **popped** off the stack, and its memory is reclaimed.
- Each call gets its **own** frame — calling the same function twice (e.g. `read_point()` for `p` then for `q` in the lab below) creates two separate frames, each with its own copy of `pt`.

```
call main()
  -> call read_point()    [frame for read_point pushed]
  <- return                [frame for read_point popped]
  -> call read_point()    [a *new* frame pushed]
  <- return                [popped again]
  -> call print_movement() [frame pushed]
  <- return                [popped]
```

This is also why a function cannot "see" another function's local variables — they live in different frames.

2.2.2 Local variables

A **local variable** is declared inside a function or block, and only exists — is only *in scope* — within that block.

```
void foo(void) {
    int a = 1; // local to foo
}

void bar(void) {
    int a = 2; // a different `a`, local to bar
}
```

`foo`'s `a` and `bar`'s `a` are unrelated; each lives in its own stack frame.

Block scoping

Any `{ }` introduces a new scope, not just function bodies — including `if`, `else`, and loop bodies:

```
int main(void) {
    int x = 10;
    if (x > 5) {
        int y = 20;     // y only exists inside this block
        printf("%d\n", y);
    }
    // y is not visible here -- out of scope
    return 0;
}
```

Lifetime vs. scope

- **Scope** — where in the code a variable's name is visible.
- **Lifetime** — how long the variable's memory actually exists (tied to its stack frame, from when the block is entered to when it's exited).

2.X Lab

More If-Else, Abstraction, Structs

Part 1a:

Does the code below compile? What is its output?

```
#include <stdbool.h>
#include <stdio.h>

bool foo(){
    printf("foo called returning true\n");
    return true;
}

bool bar(){
    printf("bar called returning false\n");
    return false;
}

int main(){
    if(foo() && bar()){
        printf("line A reached\n");
    }

    printf("BOOP\n");

    if(foo() || bar()){
```

```
        printf("line B reached\n");
    }
}
```

Output: **Only one line output**

- If you omit the curly braces `{ }` in your if statement, only the line **below** the `if` statement is used as its output.
-

Consider the following code:

```
#include "input.h"
#include <stdbool.h>
#include <stdio.h>

typedef struct {
    int x;
    int y;
} Point;

int main() {
    Point p;
    p.x = read_int();
    p.y = read_int();

    Point q;
    q.x = read_int();
    q.y = read_int();

    int dx = q.x - p.x;
    int dy = q.y - p.y;

    bool moving_right = dx > 0;
    bool moving_up = dy > 0;
    bool on_same_x = dx == 0;
    bool on_same_y = dy == 0;

    if (on_same_x && on_same_y) {
        printf("same point\n");
    } else if (on_same_x) {
        printf("vertical movement\n");
    } else if (on_same_y) {
        printf("horizontal movement\n");
    } else if (moving_right && moving_up) {
        printf("northeast\n");
    } else if (!moving_right && moving_up) {
        printf("northwest\n");
    } else if (moving_right && !moving_up) {
        printf("southeast\n");
    } else {
        printf("southwest\n");
    }
    return 0;
}
```

We can simplify this to:

```
#include "input.h"
#include <stdbool.h>
#include <stdio.h>

typedef struct {
    int x;
```

```
    int y;
} Point;

Point read_point() {
    Point pt;
    pt.x = read_int();
    pt.y = read_int();
    return pt;
}

void print_movement(Point start, Point end) {
    int dx = end.x - start.x;
    int dy = end.y - start.y;

    bool moving_right = dx > 0;
    bool moving_up = dy > 0;
    bool on_same_x = dx == 0;
    bool on_same_y = dy == 0;

    if (on_same_x && on_same_y) {
        printf("same point\n");
    } else if (on_same_x) {
        printf("vertical movement\n");
    } else if (on_same_y) {
        printf("horizontal movement\n");
    } else if (moving_right && moving_up) {
        printf("northeast\n");
    } else if (!moving_right && moving_up) {
        printf("northwest\n");
    } else if (moving_right && !moving_up) {
        printf("southeast\n");
    } else {
        printf("southwest\n");
    }
}

int main() {
    Point p = read_point();
    Point q = read_point();
    print_movement(p, q);
    return 0;
}
```

L3: Fixed Size arrays and Recursion

3.1 Arrays

3.1.1 Array Declaration

Let's say we want to declare an array that stores 5 elements of type `double` that we want to call "costs".

An **array** is declared as such: `<type> <array name>[<no. of elements>]`

```
double costs[5];
```

> **Important:** Arrays have a fixed type. You cannot add multiple different types to an array unlike Python.

Arrays are stored on the stack.

```
main

costs = {?,?,?,?,?}
```

An array will have all of its elements stored contiguously to each other in memory. * This has implications for both performance and security in the far future.

```
double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
        // initializes the array with 5 values
```

It is stored in the stack as such:

```
stack:
|8.88|10.22|9.88|22.22|44.44|
```

3.1.2 Initializing elements

However, what if we initialize only 3 elements out of 5?

```
double costs[5]= {8.88,10.22,9.88};
// the remaining elements will be initialized to 0.0 by default.


double zeroarray[5]={0};
//^^this is a very good way to initialize an entire array to 0!
```

What if we initialize more than 5 elements?

```
double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44, 1.0};
```

If we have more values than the specified size, what happens is up to the compiler (technically not undefined behavior) [we don't recommend doing this, there's no good reason to at this stage]

What if we initialize it over 2 lines?

```
double costs[5];
costs = {8.88, 10.22, 9.88, 22.22, 44.44, 1.0};
```

> This does **NOT** work and throws a `compile error`. Unlike scalar variables, arrays cannot be assigned new values as a whole.

3.1.3 Accessing an Array (writing)

We use the index.

```
double costs[5];

costs[0] = 8.88;
// | 8.88 | ? | ? | ? | ? |

costs[4] = 44.44;
// | 8.88 | ? | ? | ? | 44.44 |
```

3.1.4 Copying Semantics

```
int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
```

```
    double costs_2[] = costs; // error!
}
```

> This does **NOT** work and will throw a *compile error*! `error: array initializer must be an initializer list.`

Arrays (unlike numerical types and structs) as a whole are **not copyable** via assignment operators.

3.1.5 Arrays as arguments to functions

```
#include <stdio.h>
#include <stddef.h>

void print_doublearr_index(double arr[], size_t index) {
    printf("%f\n", arr[index]);
}

int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    print_doublearr_index(costs, 3);
}
```

`size_t`: unsigned type (typically `unsigned long`)

- Guaranteed to be large enough to index any array
- Intent is clear: use by default for array indices, unless we need negative values
- `size_t` is defined in `<stddef.h>`

Array indices can technically be any integral type (`char`, `int`, `long` ...)

3.1.6 Length of arrays

```
void print_last_element (double arr[]) {
    printf("%f\n", arr[?])
}

int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    print_last_element(costs);
}
```

In **C**, we do **NOT** have `length/len` operators to measure the length of an array.

Instead, we use `size_t` for now:

```
void print_last_element(double arr[], size_t length) {
    printf("%f\n", arr[length - 1]);
}

int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    print_last_element(costs, 5);
}
```

Setting Values inside Functions

```
int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
```

```
    set_doublearr_index(costs, 3, 5.5);
    printf("%f\n", costs[3]);
}
void set_doublearr_index(double arr[], size_t index, double val) {
    arr[index] = val;
}
    //output is... 5.5.
```

This sets the index 3 (the 4th) element of costs to 5.5. But how is that possible? This is because arrays are pointers.

Returning from functions

For now, do not return arrays declared within functions.

```
// DONT DO THIS
??? foo() {
    double costs[5] ={8.88, 10.22, 9.88, 22.22, 44.44, 1.0};
    return costs;
} // will return UB
```

3.2 Arrays and Structs

Zeroing out

```
#include <stdio.h>

typedef struct Position {
    int x;
    int y;
} Position;

int main(void){
    Position positions[2] = {0};
    printf("%d %d\n", positions[0].x, positions[0].y);
} // This will zero out all members of all structs.
```

UB

```
int main(void) {
    double costs[1] = {8.88};
    printf("%f\n", costs[1]);
}

// UB [1]


int main(void) {
    double costs[1];
    printf("%f\n", costs[-1]);
}

// UB [2]

??? foo() {
    double costs[1] = {8.88};
    return costs;
}

// UB [3]
```

```c
int main() {
    double costs[1];
    printf("%f\n", costs[0]);
}
```

Variable-length arrays

Can we declare the size of an array as a variable? Yes.

```c
double costs[var];
```

> However, there are much better ways to do this (discussed later). We do not allow VLAs because of underlying issues. As a matter of fact, VLA was removed from Linux operating system.

3.3 Multidimensional Arrays

3.3.1 Array of arrays

```c
int main(void) {
    int a[2][3] = {{0,1,2},{3,4,5}};
    printf("%d\n", a[1][0]);
}
```

Then a is stored as:

$$a = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}$$

We are storing an array of arrays.

Example: tic-tac-toe

```c
int grid[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
```

$$\text{grid} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Arrays can have more dimensions!

> Only the first dimension size can be omitted.

3.4 Recursion

3.4.1 Why Recursion

You **cannot call yourself a programmer if you don't know how to recurse**. You will be locking yourself away from future concepts if you don't learn this. This includes:

- Binary search trees
- Fast/Efficient sorting algorithms
- A lot of efficient algorithms. The fast Fourier transform (FFT) is an example

3.4.2 Features of recursion

- **Base Case:**
  - The simplest possible case that we have to solve manually.
- **Recursive Case:**
  - In this case we usually have to "steer" the general case into the smaller case somehow.
  - We will rely on Cheems to solve the smaller version of the problem for us.
  - We will use whatever Cheems gives us + a little bit more work to solve the problems in general.

```
<return_type> foo(<param(s) here>){
    if(<base case>){
    return; // or return value;
}

// handle recursive case here
// any computation before recursive call(s)

foo(<with smaller case>);
    // any computation after recursive call(s)
}
```

3.4.3 Example: Printing an array in reverse

Logic:

1. The base case is when the length of the array is 0, so we do nothing.
2. We have an entire array of n elements.
3.

- The entire array of n elements
  - The last element will be our base case.
  - Cheems will handle the rest with: `print_arr_reverse(arr, len-1)`
- We should do something only with the last element. We print the last element before Cheems.

```
void print_arr_reverse(int arr[], size_t len){
    if (len == 0) {
        return; // array has nothing, we do nothing.
    }

    printf("%d\n", arr[len-1]);

    print_arr_reverse(arr, len-1); // cheems!
}
```

Example: Reverse an array

Input: An array of length `len`. Output: Reverse the elements of `arr`.

We need a helper function here: The function will need:

- take in an array
- Swap the first and last elements

```
void reverse_arr_helper(int arr[], size_t start, size_t end) {
    if (start >= end) {
```

```
        return;
    }
    reverse_arr_helper(arr, start+1, end-1);
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
}

void reverse_arr(int arr[], size_t len) {
    if (len==0) {return;}
    reverse_arr_helper(arr,0,len-1);
}
```

Example: Sort an array

Let's say Cheems has already sorted the first $n-1$ elements for us. What should we do with the last element? > We figure out where to put it.

```
void shift_left(int arr[], size_t len, size_t count) {
    ...
}

void count_larger_than(int arr[], size_t len, size_t val) {
    ...
}

void sort_arr(int arr[], size_t len) {
    if(len==0){
        return;
    }
    size_t count = count_larger_than(arr, len-1, arr[len-1]);
    shift_left(arr, len, count);
}
```

3.L Lab

- When declaring `arrays[][]`, you need to at least declare the no. of columns `[][cols]`.
- `void` will not affect the variable inside `main` without `return`.
- However, `void` calls on the array directly and can redefine the array elements.

L4: Loops

4.1 `while` Loops and `for` Loops

Why loops?

Some problems can be solved using recursion, but recursion can use a lot of stack space. Consider printing every element of an array:

```
void print_arr_from(double arr[], size_t length, size_t i) {
    if (i == length) {
        return;
    }
    printf("%f\n", arr[i]);
    print_arr_from(arr, length, i + 1);
}

void print_arr(double arr[], size_t length) {
```

```
    print_arr_from(arr, length, 0);
}
```

Each recursive call uses a stack frame and waits for the next call to finish. Printing an array of length n creates roughly n nested calls, so a sufficiently large array can exhaust the stack and cause a **stack overflow**. C does not guarantee tail-call optimization to avoid this.

Loops let us repeat code without making a new function call for every repetition.

`while` loops

```
while (expression) {
    // loop body
}
```

The expression is checked first. If it is true, the loop body runs. At the end of the body, execution returns to the condition and checks it again. If the expression is false, the loop body is skipped or the loop ends.

For example, print every element of an array:

```
void print_arr(double arr[], size_t length) {
    size_t i = 0;

    while (i < length) {
        printf("%f\n", arr[i]);
        i += 1;
    }
}
```

Here, `i` is the **index** (or iterator). It starts at zero, increases after each iteration, and the loop ends when `i < length` becomes false.

If the loop body never changes anything that can make the condition false, the loop may run forever. For example, leaving out `i += 1` would repeatedly print `arr[0]`.

Array indices go from `0` to `length - 1`. Using `i <= length` would eventually access `arr[length]`, which is outside the array and causes undefined behavior.

For reverse iteration with `size_t`, avoid a condition such as `i >= 0`: `size_t` is unsigned, so that condition is always true, even after the value wraps around. One way to print in reverse is:

```
void print_arr_reverse(double arr[], size_t length) {
    for (size_t i = 0; i < length; i += 1) {
        printf("%f\n", arr[(length - 1) - i]);
    }
}
```

`for` loops

A `for` loop puts initialization, condition, and iteration in one place:

```
for (expression1; expression2; expression3) {
    // loop body
}
```

- `expression1` runs once before the loop starts. It often declares and initializes an index.
- `expression2` is the condition. The body runs only when it is true.
- `expression3` runs after each execution of the body. It often updates the index.

The `while` and `for` versions below do the same thing:

```
size_t i = 0;
while (i < length) {
    printf("%f\n", arr[i]);
    i += 1;
}
```

```
for (size_t i = 0; i < length; i += 1) {
    printf("%f\n", arr[i]);
}
```

Variables declared in a `for` loop's initialization are scoped to the loop, so `i` cannot be used after the loop. A variable declared before a `while` loop remains accessible after it.

4.2 Loop control with `break`, `continue`, `return`

`break`

`break` immediately exits the **innermost loop** containing it. Execution continues after that loop.

```
for (size_t i = 0; i < 5; ++i) {
    printf("%zu\n", i);
    if (i == 1) {
        break;
    }
}
printf("Loop ended!\n");
```

This prints `0`, `1`, and `Loop ended!`. In nested loops, `break` exits only the inner loop.

`continue`

`continue` skips the rest of the current loop body and moves to the next iteration. In a `for` loop, the iteration expression runs before the condition is checked again.

```
for (size_t i = 0; i < 5; ++i) {
    if (i == 1) {
        continue;
    }
    printf("%zu\n", i);
}
```

This prints `0`, `2`, `3`, and `4`.

`return`

`return` immediately exits the **entire function**, including any loops inside it. For example, this function returns as soon as it finds a `1`:

```
bool has_one_inside_array(int arr[], size_t length) {
    for (size_t i = 0; i < length; ++i) {
        if (arr[i] == 1) {
            return true;
        }
    }
    return false;
}
```

4.3 Array recursion

Array elements are commonly processed by iterating over their indices:

```
for (size_t i = 0; i < length; ++i) {
    // process arr[i]
}
```

This is useful for tasks such as printing every element, initializing an array, and calculating a sum. The index starts at `0`, and the loop condition `i < length` ensures the last index accessed is `length - 1`.

4.4 Examples

Initialize every array element

```
void init_array(double arr[], size_t length, double val) {
    for (size_t i = 0; i < length; ++i) {
        arr[i] = val;
    }
}
```

`++i`, `i++`, and `i += 1` all increase `i` by one here.

Nested loops: print a triangle

A loop can be placed inside another loop. The inner loop runs to completion during each outer loop iteration.

```
void print_triangle(size_t height) {
    for (size_t i = 0; i < height; ++i) {
        for (size_t j = 0; j < i + 1; ++j) {
            printf("X ");
        }
        printf("\n");
    }
}
```

The outer loop controls the row number. The inner loop prints `i + 1` Xs on that row. Each loop's index is scoped to its own `for` loop.

A method for planning loops

1. Decide what condition must be true for the loop to continue.
2. If the loop needs an iterator, choose its type and initialize it.
3. Decide how to update the iterator after each iteration.

L5: Pointers and Strings

5.1 Pointers

Memory

Memory has two important properties:

- **Address**: the location where data is stored.
- **Value**: the data stored at that location.

For example:

```
int x = 4;
int *p = &x;
```

An illustrative memory layout is shown below. Actual addresses, object sizes, and placement depend on the implementation.

Address	Value
0x1000	0x4
0x1004	0x1000
0x1008	
0x100C	
0x1010	
0x1014	
0x1018	
0x101C	

Pointers

A pointer variable stores an address.

```
int x = 4;
int *p = &x;
```

- `int x = 4;` declares an integer named `x` and initializes it to 4.
- `int *p = &x;` declares a pointer to `int`, named `p`, and initializes it with the *address of* `x`.
- In this declaration, `*` indicates a pointer type.
- In an expression, `&` is the address-of operator.

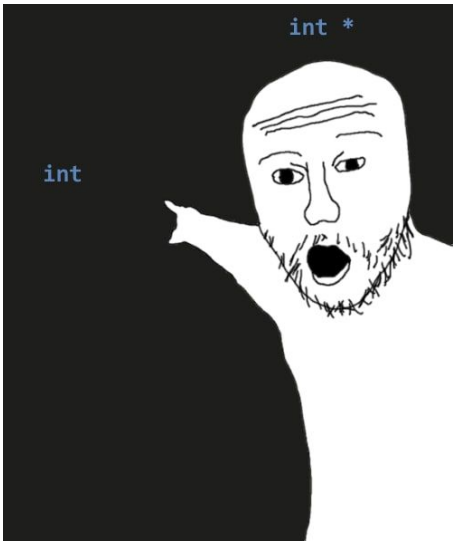


Figure 5.1.1: A pointer storing an object's address.

Pointer size: A pointer's size is determined by its pointer type and the implementation. It does *not* have to equal the size of the object it points to. For example, a pointer may occupy 8 bytes while pointing to a 4-byte `int`.

Why pointers?

A function can use a pointer to access an object declared elsewhere, provided that the object is still alive and the pointer is valid.

Passing a pointer to a struct lets the function access or modify the original struct without copying the entire struct. The pointer itself is still passed **by value**.

5.2 Address-of (&) and Dereferencing (*), nullptr, pointers to pointers

5.2.1 Printing Addresses

```
char x = 'A';
printf("x is at %p\n", (void *)&x);
```

- `%p` prints a pointer value.
- `&x` gives the address of `x`.
- `%p` expects a `void *` argument, so cast an object pointer to `void *` when printing it.

An object may occupy multiple bytes, but it still has a starting address:

```
int y = 1000;
printf("y is at %p\n", (void *)&y);
```

- `&y` has type `int *` and points to the entire object `y`.
- Casting to `char *` is useful for accessing an object's individual bytes; it is not required to print its address.
- The size of `int` is implementation-dependent.

Addresses are commonly printed in hexadecimal form:

- Example: `0x7ffd7f613f2b`.
- Hexadecimal is base 16, using the digits $0, 1, \ldots, 9, A, B, C, D, E, F$.
- This example corresponds to the decimal number 140726740533035.
- The exact format produced by `%p` is implementation-defined.
- Addresses are not guaranteed to remain the same across different executions of a program.

Takeaway: Use an expression such as `&x` to obtain an object's address; do not guess its address.

5.2.2 Address (&) and Dereferencing (*)

```
int *p = &x;
int y = *p;
```

In an expression, unary `*` performs **pointer dereferencing**: it accesses the object pointed to by the pointer.

Read `int y = *p;` as: "Initialize the integer `y` with the value of the object pointed to by `p`."

```
int x = 1;
int *ptr = &x;
printf("%d\n", *ptr);
// 1
```

- `int *ptr = &x;` declares a pointer to an integer.

- The unary operator `&` obtains an object's address.
- `*ptr` accesses the object at that address.
- `*ptr = 1;` sets the pointed-to object to 1.

```c
int main(void) {
    int x = 5;
    int *y = &x;

    printf("%d\n", x); // 5

    // Go to the object pointed to by y and set its value to 10.
    *y = 10;

    printf("%d\n", x); // 10
}
```

5.2.3 nullptr

- `nullptr` is a null pointer constant introduced in C23. It can initialize a pointer that points to no valid object or function.
- Dereferencing a null pointer causes **undefined behaviour**.
- You can test whether a pointer is null using `ptr == nullptr`.
- Being non-null alone does not guarantee that a pointer is valid to dereference.

```c
int main(void) {
    int *y = nullptr;
    // Dereferencing y would cause undefined behaviour.
}
```

5.2.4 Pointer to a pointer

- A pointer variable is itself stored in memory and has an address, so another pointer can point to it.
- If `z` points to `y`, and `y` points to `x`, then `**z` accesses `x`.

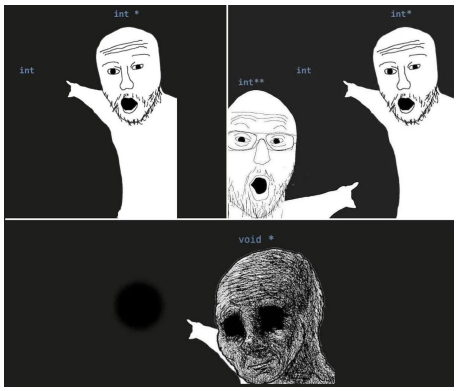


Fig. 5.2.4. A pointer to another pointer.

```c
int main(void) {
    int x = 5;
    int *y = &x;
    int **z = &y;

    // *z is y: a pointer to int.
    // **z is x: an int with value 5.
}
```

Summary: A pointer variable stores an address.

- Declare a pointer: `<type> *<ptr_name>;`
- Access the pointed-to object: `*<ptr_name>`
- Obtain a variable's address: `&<var_name>`
- Assign to the pointed-to object: `*<ptr_name> = <value>;`

5.3 Arrays, Pointer Arithmetic, `sizeof`, Structs, Functions

5.3.1 Array Memory Addresses

Arrays and pointers are **not identical**, but array expressions often undergo a conversion called **array-to-pointer decay**.

- In most expressions, an array is converted to a pointer to its first element, at index 0.
- Important exceptions include an array used as the operand of `sizeof` or unary `&`.
- In a function parameter declaration, array notation is adjusted to pointer notation.

These two function declarations describe the same parameter types:

```
void set_doublearr_index(double arr[], size_t index, double val);

// Equivalent declaration:
void set_doublearr_index(double *arr, size_t index, double val);
```

In the function call below, `costs` is converted to `&costs[0]`, the address of its first element.

```
void set_doublearr_index(double arr[], size_t index, double val) {
    arr[index] = val;
}

int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    set_doublearr_index(costs, 3, 5.5);
    printf("%f\n", costs[3]);
}
```

Array elements are guaranteed to occupy consecutive locations in memory. Arrays are not necessarily stored on a stack.

```
int main(void) {
    char chararr[4] = {0};
    int intarr[4] = {0};

    printf("chararr[0]: %p vs chararr[1]: %p\n\n",
           (void *)&chararr[0], (void *)&chararr[1]);

    printf("intarr[0]: %p vs intarr[1]: %p\n",
           (void *)&intarr[0], (void *)&intarr[1]);
}
```

Adjacent `char` elements are one byte apart. Example output:

```
chararr[0]: 0x7ffe74df3b2c vs chararr[1]: 0x7ffe74df3b2d
```

Adjacent `int` elements are `sizeof(int)` bytes apart. On a system where `sizeof(int) == 4`, example output is:

```
intarr[0]: 0x7ffe74df3b10 vs intarr[1]: 0x7ffe74df3b14
```

5.3.2 Pointer arithmetic

- Indexing with `a[i]` is equivalent to `*(a + i)`.
- One operand of this addition is a pointer; the other has integer type.
- `x + i` points i elements after `x`.
- `*(x + i)` accesses the object at that position.
- Pointer arithmetic must stay within the same array or reach at most one position past its end. The one-past pointer must not be dereferenced.

Incrementing pointers (Lab 5)

Refer to Lab 5, activity 1.

Each function receives its own copy of the pointer `arr`, initially pointing to `arr[0]`. Moving that local pointer does not change the array; modifying `*arr` changes an actual array element.

Expression	Parsed as	Effect
`++arr`	`++arr`	Moves the pointer to the next element.
`++*arr`	`++(*arr)`	Increments the pointed-to integer.
`*arr++`	`*(arr++)`	Dereferences the old pointer and advances the pointer to the next element; the integer is unchanged.
`(*arr)++`	`(*arr)++`	Increments the pointed-to integer.

5.3.3 `sizeof`

- `sizeof` determines the size, in bytes, of a type or the type of an expression.
- For ordinary non-variable-length types, its result is determined at compile time. Variable-length arrays can require evaluation at runtime.
- **Return type**: `size_t`, an unsigned integer type.
- Print a `size_t` value using `printf` with `%zu`.

```c
int num = 42;

printf("Size of int: %zu bytes\n", sizeof(int));
printf("Size of char: %zu byte\n", sizeof(char)); // Always 1
printf("Size of num: %zu bytes\n", sizeof num);

int my_array[] = {10, 20, 30, 40, 50};

size_t total_bytes = sizeof(my_array);
size_t array_length = sizeof(my_array) / sizeof(my_array[0]);

printf("Total memory size: %zu bytes\n", total_bytes);
printf("Number of elements: %zu\n", array_length);
```

Important: The array-length calculation works when `my_array` is an actual array. It does not work on a pointer or on an array parameter that has been adjusted to a pointer.

Important array–pointer relationship

Arrays and pointers are not equivalent.

This causes a compilation error:

```c
double arr1[5] = {0};
double arr2[] = arr1; // Invalid array initialization
```

However, this works:

```c
double arr1[5] = {0};
double *arr2 = arr1;
```

Here, `arr2` points to the first element of `arr1`; it is not a separate copy of the array.

5.3.4 Structs

- A **pointer to a struct** stores the address of a struct variable.
- Declare it using the struct's type followed by `*`, and initialize it with `&`.

```c
typedef struct {
    double x, y, z;
} Acceleration;

Acceleration a = {0.2, 0.3, -0.5};
Acceleration *p = &a;
```

Here, `p` points to `a`; dereferencing `p` gives access to that struct.

Accessing struct members: `.` versus `->`

Expression	Meaning
`a.x`	Access member `x` of struct `a`.
`(*p).x`	Dereference `p`, then access member `x`.
`p->x`	Shorthand for `(*p).x`.

```c
printf("%f\n", a.x);    // 0.200000
printf("%f\n", (*p).x); // 0.200000
printf("%f\n", p->x);   // 0.200000

p->x = 1.5;
printf("%f\n", a.x);    // 1.500000: a itself was modified.
```

- Use **`.` with a struct object** and **`->` with a pointer to a struct**.
- Parentheses are needed in `(*p).x` because `.` has higher precedence than unary `*`.
- `sizeof p` gives the size of the pointer; `sizeof *p` gives the size of the struct.

Why pass structs using pointers?

- A function can modify the caller's original struct through its pointer.
- Passing a pointer avoids passing the entire struct by value, which can reduce copying for large structs.
- It does not guarantee faster execution; compiler optimizations and the task affect performance.
- For a function that should only read the struct, use a parameter such as `Acceleration const *p`.

5.3.5 Functions

- C passes arguments **by value**: each parameter receives a copy of its argument.
- Passing a pointer copies the **pointer value**, not the pointed-to object.
- Dereferencing that copied pointer accesses the caller's object, allowing the function to modify it.

Passing a value versus passing its address

```c
#include <stdio.h>
```

```c
void set_copy(int x) {
    x = 10; // Changes only the local copy.
}

void set_ptr(int *p) {
    *p = 10; // Changes the caller's integer.
}

int main(void) {
    int n = 5;

    set_copy(n);
    printf("%d\n", n); // 5

    set_ptr(&n);
    printf("%d\n", n); // 10
}
```

Step	Meaning
`set_copy(n)`	Copy the value of `n` into the parameter `x`.
`set_ptr(&n)`	Copy the address of `n` into the parameter `p`.
`*p = 10`	Write `10` into the object pointed to by `p`, which is `n`.

Example: correcting a struct

Using the `Acceleration` type from 5.3.4:

```c
// Pass the struct by value and return the corrected copy.
Acceleration correct_copy(Acceleration a) {
    a.x -= 0.00000002;
    a.y += 0.00000001;
    a.z -= 0.00000003;
    return a;
}

// Pass its address and modify the original struct directly.
void correct_ptr(Acceleration *a) {
    a->x -= 0.00000002;
    a->y += 0.00000001;
    a->z -= 0.00000003;
}
```

Approach	Call in `main`	Effect
Pass the struct by value	`a = correct_copy(a);`	Return a corrected copy and assign it back to `a`.
Pass the struct's address	`correct_ptr(&a);`	Modify the original `a` through its address.

- Calling `correct_copy(a)` without assigning its return value leaves the caller's `a` unchanged.
- `correct_ptr` returns `void` because its result is written into the original struct.

Changing the object versus changing the local pointer

```c
void redirect(int *p, int *q) {
    p = q; // Changes only this function's local pointer.
}

int main(void) {
    int x = 5;
    int y = 10;
```

```
    int *ptr = &x;

    redirect(ptr, &y);
    // ptr still points to x.
}
```

- Assigning to `*p` changes the pointed-to object.
- Assigning to `p` changes only the function's local pointer.
- To change the caller's pointer variable, pass its address using a **pointer to a pointer** (`int **`); see 5.2.4.

Arrays as function parameters

- `void f(int arr[])` and `void f(int *arr)` declare the same parameter type.
- When passed an array, the function receives a pointer to its first element; it can modify elements using `arr[i]`.
- Pass the array length separately, for example: `void f(int *arr, size_t len)`.
- Inside such a function, `sizeof arr` gives the pointer size, so it cannot determine the original array length.

The caller must supply a pointer to a suitable live object; dereferencing an uninitialized, null, or dangling pointer causes **undefined behaviour**.

5.4 Strings

5.4.1 Strings and the null character

A string is a contiguous sequence of characters terminated by, and including, the first null character, `'\0'`.

N	$N+1$	$N+2$	$N+3$	$N+4$	$N+5$
`'H'`	`'e'`	`'l'`	`'l'`	`'o'`	`'\0'`

- C strings do not carry a separate length field; string functions locate the end using the null character.
- The null character acts as the string terminator.
- If an array lacks a null terminator, operations that expect a string may read beyond the array's bounds, causing **undefined behaviour**.
- This does not necessarily cause a stack overflow.

5.4.2 Modifiable and read-only strings

A modifiable character array

- The following are alternative declarations with the same initial contents. Use one declaration at a time.
- The array can be modified, for example with `p[0] = 'a';`.
- A string literal includes an implicit `'\0'`.
- To store the complete string in an array, allow room for its null terminator.

```
// Alternative declarations: use one at a time.
char p[] = "hi";
char p[] = {'h', 'i', '\0'};
char p[3] = "hi";
```

A pointer to a string literal

```
char *p = "hi";
```

- Here, `p` points to a string literal.
- Attempting to modify the literal, for example using `p[0] = 'a';`, causes **undefined behaviour**.
- Implementations often store string literals in read-only memory.
- Prefer `const char *p = "hi";` to express that the characters must not be modified through `p`.

5.4.3 Printing strings

```
const char *p = "Hello\n";
printf("%s\n", p);
```

- **Format specifier**: `%s` prints a string.
- Printing stops at the first `'\0'`; the null character itself is not printed.
- This example prints two newline characters: one from the string and one from the format string.

5.4 Strings

5.4.1 Strings and the null character

A string is a contiguous sequence of characters terminated by, and including, the first null character, `'\0'`.

N	$N+1$	$N+2$	$N+3$	$N+4$	$N+5$
`'H'`	`'e'`	`'l'`	`'l'`	`'o'`	`'\0'`

- C strings do not store a separate length field. String functions find the end using the null character.
- The null character acts as the string terminator.
- Without a terminator, operations that expect a string may read beyond the array's bounds, causing **undefined behaviour**.

5.4.2 Modifiable and read-only strings

A modifiable character array

- The following are alternative declarations with the same initial contents. Use one declaration at a time.
- The array can be modified, for example using `p[0] = 'a';`.
- String literals automatically include an extra `'\0'`.
- To store the complete string, the array must have space for the null terminator.

```
// Alternative declarations: use one at a time.
char p[] = "hi";
char p[] = {'h', 'i', '\0'};
char p[3] = "hi";
```

A pointer to a string literal

```
char *p = "hi";
```

- Here, `p` points to a string literal.
- Attempting to modify the literal, such as with `p[0] = 'a';`, causes **undefined behaviour**.
- Implementations often store string literals in read-only memory.

5.4.3 Printing strings

```
char *p = "Hello\n";
printf("%s\n", p);
```

- **Format specifier**: `%s` prints a string.
- Printing stops at the first `'\0'`, which is not printed.
- This example prints two newline characters: one from the string and one from the format string.

5.4.4 `const`

- `const` is a **type qualifier** indicating that something must not be modified through the const-qualified access.
- For pointers, its position determines whether the pointed-to value, the pointer itself, or both are protected.
- Use it when a value or function input should not be modified. It communicates intent and helps the compiler catch mistakes.

```
int const x = 5; // Equivalent to: const int x = 5;
// x = 10;      // Compiler error: x is const.
```

`const` and string literals

Modifying a string literal is **undefined behaviour (UB)**, even when the pointer has type `char *`.

```
char *p = "hi";
// p[0] = 'a'; // UB: attempts to modify a string literal.
```

Prefer a pointer to `const char` so the compiler catches attempts to write through it:

```
char const *p = "hi"; // Equivalent to: const char *p = "hi";

// p[0] = 'a';       // Compiler error: cannot modify through p.
p = "bye";           // Allowed: the pointer itself can change.
```

A pointer to a const value versus a const pointer

The position of `const` matters. Here, assume that `x` is a modifiable `int`.

Declaration	Meaning	Reassign p?	Assign to *p?
`int *p = &x;`	Pointer to an integer	Yes	Yes
`int const *p = &x;`	Pointer to a const integer	Yes	No
`int *const p = &x;`	Const pointer to an integer	No	Yes
`int const *const p = &x;`	Const pointer to a const integer	No	No

- `const` **before** `*` qualifies the pointed-to type.
- `const` **after** `*` qualifies the pointer itself.
- `int const *p` and `const int *p` are equivalent.

`const` does not make every object it points to unchangeable

A pointer to const prevents modification **through that pointer**. A modifiable object can still be changed through another name or pointer.

```
char text[] = "hi";
char const *p = text;

// p[0] = 'H'; // Compiler error.
text[0] = 'H'; // Allowed: text is a modifiable array.

printf("%s\n", p); // Hi
```

However, if the object itself was declared `const`, attempting to modify it, even through a cast, causes UB.

`const` in function parameters

```
void print_arr(int const arr[], size_t len);

// Equivalent parameter type:
void print_arr(int const *arr, size_t len);
```

- The function can read `arr[i]`, but cannot assign to `arr[i]` through this parameter.
- The local pointer can still move, for example using `++arr`.
- The caller can pass a modifiable array. Passing it does not make the original array const.

Examples from `<string.h>` using simplified declarations:

Function declaration	Meaning of `const`
`size_t strlen(char const *s);`	Reads the string and returns its length, excluding `'\0'`.
`char *strcpy(char *dest, char const *src);`	Copies `src`, including `'\0'`, into writable `dest`.

For `strcpy`, `dest` must have enough space, and the source and destination must not overlap.

5.5 Pointer Provenance

A pointer's address alone does not tell us whether using it is valid. We also need to know **where it came from**, **what memory it can access**, and **whether that memory still exists**.

The lecture uses pointer provenance as a **mental model** with two components:

Component	Question
Spatial	Which memory locations can this pointer access?
Temporal	During what part of the program's execution can it access them?

These are facts we must reason about; ordinary C pointers do not automatically store an array length or expiry time.

5.5.1 Origins: what does the pointer represent?

Two pointers with the same type can represent different things:

```
char ch = 'b';
char text[] = "abcd";

char *p = &ch;  // Points to one character.
```

```
char *q = text; // Points to the first character of a string.
```

- `*p` is valid, but `p` does not point to a null-terminated string. Do not pass it to `%s` or `strlen`.
- `q` points into an array containing `{'a', 'b', 'c', 'd', '\0'}`, so it can be used as a string.
- Similarly, `char **r` may point to one `char *` variable or to an element of an array of `char *` pointers.
- The type alone does not tell us the array length, whether there is a string terminator, or whether the contents are writable.

Takeaway: Trace how the pointer was created and what the caller promises before deciding how to use it.

5.5.2 Spatial component: access within valid bounds

For an array of n elements:

- Dereference only pointers to actual elements: indices 0 to $n - 1$.
- Pointer arithmetic may produce a pointer within the same array, or **one past its end**.
- A pointer one past the end may be formed, but **must not be dereferenced**.
- Forming a pointer farther outside the array using pointer arithmetic is already UB.

```
int arr[] = {10, 20, 30};
int *p = arr;

printf("%d\n", p[2]); // 30: valid element.
int *end = arr + 3;   // Valid: one past the end.

// printf("%d\n", *end); // UB: no element exists here.
// printf("%d\n", p[-1]); // UB: before the array.
// int *bad = p + 4;     // UB: beyond one past the end.
```

- Knowing that another variable is nearby in memory does not allow accessing it by indexing beyond this array.
- Subtracting pointers requires both to refer to the same array, or one past it. The result counts **elements**, not bytes, and must be representable in `ptrdiff_t`.

Spatial reasoning for strings

```
void print_str(char const *ptr) {
    while (*ptr != '\0') {
        putchar(*ptr++);
    }
}

int main(void) {
    char const s[] = "halp";
    print_str(s);
}
```

- The caller must provide a valid string with an accessible `'\0'`.
- `*ptr++` means `*(ptr++)`: it yields the character at the old pointer position and advances the pointer.
- The loop checks the terminator and stops without reading beyond it.
- A larger character array may contain unused space after `'\0'`. The string ends at the first null character, while the array's bounds depend on its size.

5.5.3 Aliasing and overlapping ranges

Aliasing means different pointers refer to the same object.

```
int value = 0;
int *x_ptr = &value;
int *y_ptr = &value;

*x_ptr = 5;
*y_ptr = 20;

printf("%d\n", *x_ptr); // 20: both point to value.
```

Writing through one pointer can therefore change what another pointer reads.

Example: copying an array without modifying the source

```
void copy_arr(int const from[], int to[], size_t nmemb) {
    for (size_t idx = 0; idx < nmemb; ++idx) {
        to[idx] = from[idx];
    }
}
```

To copy correctly while leaving the source unchanged, use the following contract:

- `from` has at least `nmemb` readable elements.
- `to` has at least `nmemb` writable elements.
- The ranges `[from, from + nmemb)` and `[to, to + nmemb)` do not overlap. The closing `)` excludes the endpoint.

What if the caller passes overlapping parts of one array?

```
int arr[] = {1, 2, 3, 4};

copy_arr(arr, arr + 1, 3);

// Result: {1, 1, 1, 1}, rather than the intended {1, 1, 2, 3}.
```

- Writing `to[0]` changes `from[1]` before it is read. Subsequent iterations propagate that changed value.
- The const-qualified element type prevents writes **through `from`**, but does not prevent writes to the same elements through `to`.
- This loop's overlapping accesses are not automatically UB. This call violates the stated copying contract and gives the wrong result.

5.5.4 Temporal component: the pointed-to object must still exist

An object's **lifetime** is the period during which it exists. A pointer can only access it during that period.

Returning a pointer to a local array

```
int *get_arr(void) {
    int arr[] = {0, 1, 2};
    return arr; // Bad: arr's lifetime ends when the function returns.
}

int main(void) {
    int *p = get_arr();
    printf("%d\n", p[0]); // UB: the local array no longer exists.
}
```

- Returning the pointer does not copy the array or extend its lifetime.
- The pointer is now a **dangling pointer**: its former object no longer exists.
- Memory that appears to retain the old values does not make accessing it valid.

Leaving a block

```
int *p;

{
    int x = 10;
    p = &x;
} // x's lifetime ends here.

// printf("%d\n", *p); // UB: x no longer exists.
```

The pointer variable may still exist even after the pointed-to object has stopped existing.

5.5.5 Uninitialized pointers and pointers to pointers

Declaring a local pointer does not create an object for it to point to:

```
int *p;

// *p = 5; // UB: p is uninitialized; it has no valid target.
```

Initialize it with the address of a suitable live object before dereferencing:

```
int x;
int *p = &x;

*p = 5; // Valid: writes to x.
```

However, we can pass the **address of an uninitialized pointer variable** to a function that assigns it a value:

```
void foo(int **x) {
    *x = nullptr; // C23: assigns a null pointer to the caller's p.
}

int main(void) {
    int *p;
    foo(&p);

    // p is now a null pointer; dereferencing p would be UB.
}
```

- `&p` is valid because the pointer variable `p` itself exists.
- `x` points to `p`, so `*x = nullptr;` gives `p` a defined value.
- The function does not read the old value of `p` or dereference that old value.

Before using a pointer, check: its origin, valid bounds, the object's lifetime, permission to read or write, and whether other pointers access the same memory.

L6: Dynamic memory allocation

A pointer is a separate object from the object it points to.